Mixed-mode Information Flow Tracking with Compile-time Taint Semantics Extraction and Offline Replay

ABSTRACT

Static information flow analysis (IFA) and dynamic information flow tracking (DIFT) have been widely employed in offline security analysis of computer programs. As security attacks become more sophisticated, there is a rising need for IFA and DIFT in production environment. However, existing systems usually deal with IFA and DIFT separately, and most DIFT systems incur significant performance overhead. We propose MIT to facilitate IFA and DIFT in online production environment. MIT offers mixed-mode information flow tracking at byte-granularity and incurs moderate runtime performance overhead. The core techniques consist of the extraction of taint semantics intermediate representation (TSIR) at compile-time and the decoupled execution of TSIR for information flow analysis. We conducted extensive performance overhead evaluation on MIT to confirm its applicability in production environment. We also outline potential applications of MIT including the implementation of data provenance checking and information flow based anomaly detection in real-world applications.

CCS CONCEPTS

• **Security and privacy** → **Software security engineering**; • **Software and its engineering** → *Software reliability*; *Software fault tolerance*; *Software safety*.

KEYWORDS

Decoupled dynamic information flow tracking, Static information flow tracking, Taint propagation, Application logic vulnerabilities, Anomaly detection

1 INTRODUCTION

As today's computing applications are getting more sophisticated, it is crucial to ensure that sensitive information is securely and properly handled by the applications. Static information flow analysis (IFA) help us understand the likely flow paths of information in a program so that we can verify their consistency with the security policies. Yet, the inherent dynamisms in application workload make it rather difficult to precisely determine the security attributes of the flow paths beforehand. Dynamic information flow tracking (DIFT) address the issue by monitoring the actual flows of information at runtime. However, the monitoring process can incur significant performance overhead, and their uses are traditionally limited to offline program analysis.

In light of the increasing complexities in both the computing systems and the security attacks faced by the systems, we believe there is a pressing need for an integrated information flow analysis and tracking mechanism for production environment. We propose MIT as a solution to address the need. MIT offers a streamlined process for IFA and DIFT. MIT extracts taint semantics from the target program at compile-time and translates them into the taint semantics intermediate representation (TSIR). IFA can be carried out on the TSIR independently of the underlying architectures and languages. MIT also instruments the target program for control flow logging and runtime value logging during program execution. DIFT is carried out asynchronously by replaying the TSIR based on the runtime logs so the program execution will not be slowed down by the DIFT process. MIT, itself, also utilizes the integrated IFA process to avoid unnecessary runtime logging on irrelevant flow paths. The target program can use the runtime tag query APIs to interact with the DIFT process.

[Contributions] MIT intertwines IFA and DIFT of a target program via extraction of TSIR at compile-time and replay of TSIR to reconstruct the dynamic information flow. MIT leverages the compiler's ability in allocating the memory variables to minimize the amount of runtime logging for memory address calculation. This also allows a simple and straightforward replay process (as opposed to the use of symbolic execution [17][16]). MIT enables flexible IFA and DIFT in production environment with moderate performance overhead.

2 MIXED-MODE INFORMATION FLOW TRACKING

We propose a information flow tracking mechanism for production environment. The mechanism minimizes the runtime overhead of dynamic information flow tracking by decoupling the tracking logic from program execution. The decoupling process consists of compile-time extraction of language-independent TSIR from the target program and offline replay of the TSIR. The TSIR streamlines static information flow analysis while the replay enables dynamic information flow analysis. As illustrated in Figure 1, the mechanism consists of three phases.

Taint Semantics Extraction. Based on the LLVM toolchain , the TSIR Translator extracts taint semantics from the LLVM IR of the target program at compile-time and translates them into an intermediate representation for information flow tracking (*i.e.* the TSIR). Static information flow analysis can be directly carried out on the TSIR. In addition, the TSIR Translator instruments the LLVM IR with runtime logging code. The instrumented LLVM IR will be compiled and then get linked with the IFT runtime library to produce the instrumented target program ready for dynamic information flow tracking.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACMSE'20, March 2020, Tampa, FL, USA

© 2024 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

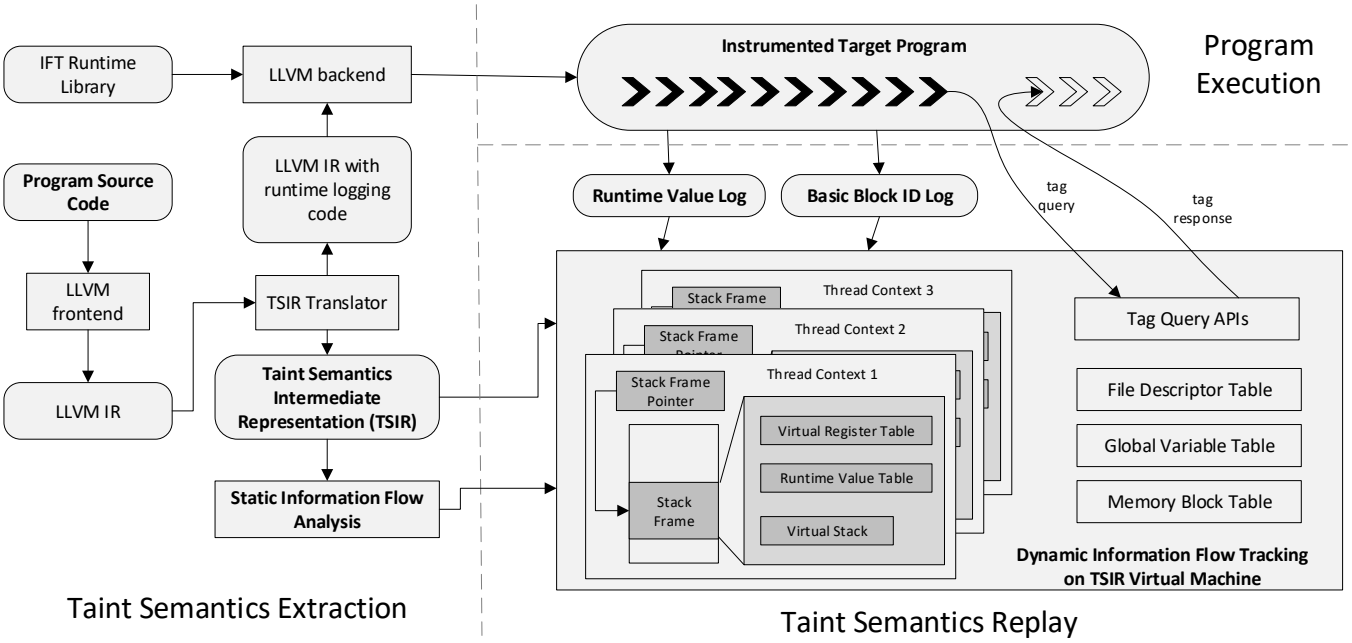

Figure 1: Mixed-mode Information Flow Tracking

Program Execution. The instrumented target program is deployed in the production environment and gets executed as usual. The instrumented runtime logging code will log the IDs of the executed basic blocks and the accompanying runtime values. Unlike typical dynamic information flow tracking, we do not perform the information flow tracking logic (*e.g.* taint propagation) inline with the program execution.

Taint Semantics Replay. Dynamic information flow tracking (DIFT) is instantiated by reconstructing the control flows and the data flows of the target program, which is achieved by replaying the TSIR with respect to the basic block ID log and the runtime value log on the TSIR virtual machine. As a result, the dynamic information tracking process can be completely decoupled from the program execution. For applications that require the program execution to wait for the result from DIFT (*e.g.* getting the data sources for a outbound network transmission), MIT provides the tag query APIs that allow the target program to interact with the DIFT process.

2.1 Taint Semantics Intermediate Representation

We propose the intermediate representation, TSIR, to describe the semantics of taint propagations and pointer passing (pass-by-address) in the target program. In one sense, it can be seen as a simplified version of LLVM IR, where only semantics relevant to information flow analysis are retained. TSIR enables language-independent static information flow analysis. At the same time, it is also specifically designed to work with dynamic information flow tracking. The incorporation of the LLVM toolchain alleviates us from the difficulties in tracking the effective addresses of memory variables [9][17][16]. In the following, we will introduce the TSIR language and the semantics represented by each TSIR instruction.

2.1.1 Taint Tags and Pointers. In LLVM IR, program data either reside on registers or in memory. A LLVM register can hold both primitive-type data such as integers and aggregate-type data such as `array`, `vector`, and `struct`. For DIFT, we are not interested in the actual content of the data. Instead, we associate tags with the data and track the flow of tags. In TSIR, we use a *virtual register* to keep the tags associated with the data of a LLVM register. A LLVM register can also hold a memory pointer, and we will soon introduce how the TSIR VM maintains logical memory block records for the memory allocations/deallocations made by the target program. As a result, a virtual register can also hold a TSIR pointer pointing to the respective location in the logical memory block records and its associated tags. In addition, corresponding to an aggregate-type LLVM register, a virtual register in TSIR can also have multiple fields of tags and pointers as shown in Table 1.

For data in the memory, The TSIR VM of MIT maintains a logical *memory block* record for each memory allocation by the target program. A memory block corresponds to a continuous memory space. It maintains the tags for the data in the memory space. As shown in Table 2, each memory block has an unique block ID. Distinct from previous work, we do not maintain per-byte shadow memory. Instead, the memory block record only keeps the meta data and is allocated upon the first access to the corresponding memory space (*i.e.* it employs lazy allocation). A memory block consists of a sequence of neighboring *memory chunks*. Each memory chunk (`offset, size, IFT record`) corresponds to a respective data variable or pointer variable in the space covered by the memory block.

2.1.2 Language Design. Similar to LLVM IR, TSIR also consists of *functions*, *basic blocks*, and *instructions*. The TSIR Translator typically maps each LLVM basic block into one corresponding TSIR basic block.

In TSIR, there are three different kinds of identifiers, including local identifier (`%n`), global identifier (`@name`), and runtime identifier (`$n`). Local identifiers are used to denote virtual registers with function scope while global identifiers are used to denote virtual registers with global scope. Runtime identifiers are used in the logging of runtime values and they always have function scope.

To access a specific field in an aggregate-type virtual register, TISR provides *register region* syntax (`%n, i`) to denote the `i`-th field of register `%n`. In addition, TSIR provides the *memory region* syntax `*%p` to denote the entire memory block pointed by the pointer-type register `%p` and the syntax (`*%p, len, off`) to denote the `len` bytes sized region starting at offset `off` in the memory block pointed by `*%p` .

As shown in Table 4, there are a total of 24 TSIR instructions. The `TAG`-prefix instructions are related to taint tag propagation and manipulation, and the `PTR`-prefix instructions are for the manipulation of pointers. Among them, instructions with `COND`-suffix are used for conditional propagation; instructions with `PHI`-suffix are used for control-flow based propagation, where the propagation depends on the ID of the predecessor basic block of the current basic block. We use the notation $T(\%1)$ to denote `%1` as a taglist and $P(\%2)$ to denote `%2` as a pointer.

2.2 Taint Semantics Extraction and Static Information Flow Analysis

The TSIR Translator extracts taint semantics from the LLVM IR of the target program and stores the semantics in TSIR form. There are four main categories of LLVM IR, including binary operations; vector & aggregate operations; memory access & addressing operations; miscellaneous operations. In the following, we give the examples on the extraction of taint semantics for binary operations and memory access & addressing operations. The processing for the other two categories follow similar steps. Finally, we will also demonstrate how we apply static information flow analysis on the TSIR to optimize away unnecessary flows.

2.2.1 Binary Operations. In LLVM IR, binary operation instructions require two operands and produce a single value. Take `add` instruction as an example. Table 3 gives the corresponding TSIR generated for each of the three usages of `add`. For the case where both operands are variables, it is translated as a `TAGUNION` instruction, that is, the destination virtual register will hold the union of the taglists from the source operands. If one of the operands is a constant, the instruction is translated as a `TAGCOPY`. The last case is the rare one, because it should have been optimized away by the compiler before entering the TSIR Translator. When both operands are constants, a `TAGCLEAR` instruction is generated to clear all taint tags on the destination register.

2.2.2 Memory Access and Addressing Operations. In LLVM, the `alloca` instruction is used for allocating memory on the stack frame. For efficiency, we do not require the instrumented target program to log the effective address of the allocated memory buffer at execution time. Instead, a `MEMALLOC_STACK` instruction is generated in the TSIR to give the information flow tracking engine a hint to assign a new memory block ID for the allocation.

The `load` and `store` instructions are used for reading from memory and writing to memory. The taint semantics of them depend on whether they are accessing data or pointer values. Table 5 shows some of the possible cases.

To *read* or *write* a specific field in an aggregate-type object in memory, the idiomatic way in LLVM IR is to first calculate the memory address of the target field with `getelementptr` instruction, and then perform `load` or `store` on that address. For example, array indices are interpreted as offset values for `getelementptr`. LLVM will also calculate the offset values for `struct` members. Some of the the offset values are statically determined by LLVM (e.g. those for `struct` members). We can just encapsulate the statically determined offset values in `PTR`-preifx instructions. For the remaining cases, where the offset values are variables (*i.e.* the values are determined at runtime), the TSIR Translator will instrument logging code in the target program to record the offset values during program execution for replay.

2.2.3 Translator and TSIR Optimizations. In LLVM IR, there are some data flows which only affect subsequent control flows. In terms of explicit information flow tracking, this kind of data flows can be ignored. We perform reachability analysis on the TSIR in each function to identify instructions that are not involved in propagating data to any memory variables or any other functions. These instructions can be pruned away to reduce the overhead of runtime value logging and replay.

The data granularity in DIFT has significant impact on the runtime time overhead of MIT when there are frequent memory accesses that involve variable offsets. We notice that this is largely due to high intensity of iterative operations on the elements of a data buffer or a string, many of which can actually be treated as a singleton in DIFT (*i.e.* using a single tag for the whole buffer, instead of keeping a separate tag for each element). For instance, leaking a few bytes of a 256-bit secret key can be seen as if the whole key is leaked. The TSIR Translator provides an option for setting the threshold size of singleton buffer. The translator will treat any `char` or `int` array (in LLVM IR, a `char` array is represented as an `i8` array), whose size is larger than the threshold as an singleton (the default threshold is 128 bytes). Taint propagation to any part of a singleton buffer is treated as if the whole buffer is getting tainted. This optimization implies that the variable offsets involved in `getelementptr` can be completely ignored in the runtime logging.

We define four log levels for developers to choose the trade-offs between analysis precision and logging overhead as shown in Table 6. Level 4 performs the most fine-grained runtime logging. Variable offsets in `getelementptr` instruction and runtime conditions in `select` instruction are all logged to achieve byte-level DIFT granularity for all data objects. In level 2 and level 3, variable offsets in `getelementptr` are logged only for non-singleton buffers. In level 1 and level 2, the runtime conditions in `select` are logged only when pointer operands are present, and `select` instructions with non-pointer operands are translated as `TAGUNION` instead of `TAGCOPY_COND` to further reduce the logging overhead.

Table 1: Virtual Register Types

Virtual Register		
Type		**Content**
Taglist		$\{tag_1, tag_2, ..., tag_n\}$
Pointer		(*mbid, offset, size, taglist*)
	0	$\{tag_1, tag_2, ..., tag_n\}$
	1	(*mbid, offset, size, taglist*)
AggType	2	—
	–	—
	n	—

Table 2: Memory Block

Memory Block		
Memory Block ID		
Offset	**Size**	**IFT Record**
0	4	$\{tag_1, tag_2, ..., tag_n\}$
8	8	(*mbid, offset, size, taglist*)
—	—	—

Table 3: TSIR Mappings for Binary Operations

LLVM IR	TSIR
`%3 = add i32 %1, %2`	`TAGUNION %3, %1, %2`
`%2 = add i32 %1, 3`	`TAGCOPY %2, %1`
`%1 = add i32 2, 3`	`TAGCLEAR %1`

Table 4: Semantics of TSIR Instructions

Instruction	Semantics
TAGCOPY *dest*, *src*	$T(dest) \leftarrow T(src)$
TAGCOPY_COND *cond*, *dest*, *src*1, *src*2	$\begin{cases} T(dest) \leftarrow T(src1), & \text{if } cond = 1 \\ T(dest) \leftarrow T(src2), & \text{otherwise} \end{cases}$
TAGCOPY_PHI *dest*, (bbid1, *src*1), (bbid2, *src*2)	$\begin{cases} T(dest) \leftarrow T(src1), & \text{if } prev_b bid = bbid1 \\ T(dest) \leftarrow T(src2), & \text{if } prev_b bid = bbid2 \\ T(dest) \leftarrow \emptyset, & \text{otherwise} \end{cases}$
TAGUNION *dest*, *src*1, *src*2	$T(dest) \leftarrow T(src1) \cup T(src2)$
TAGAPPEND *dest*, *tag*	$T(dest) \leftarrow T(dest) \cup \{tag\}$
TAGCLEAR *dest*	$T(dest) \leftarrow \emptyset$
PTRCOPY *dest*, *src*	$P(dest) \leftarrow P(src)$
PTRCOPY_COND *cond*, *dest*, *src*1, *src*2	$\begin{cases} P(dest) \leftarrow P(src1), & \text{if } cond = 1 \\ P(dest) \leftarrow P(src2), & \text{otherwise} \end{cases}$
PTRCOPY_PHI *dest*, (bbid1, *src*1), (bbid2, *src*2)	$\begin{cases} P(dest) \leftarrow P(src1), & \text{if } prev_b bid = bbid1 \\ P(dest) \leftarrow P(src2), & \text{if } prev_b bid = bbid2 \\ P(dest) \leftarrow null, & \text{otherwise} \end{cases}$
PTRASSIGN *dest*, *src*, size, offset, ($1, 4)	$P(dest) \leftarrow (P(src).mbid, P(src).offset + offset + \$1 * 4, size, P(src).taglist)$
PTRCAST *dest*, *src*, size	$P(dest) \leftarrow (P(src).mbid, P(src).offset, size, P(src).taglist)$
MEMALLOC_STACK *dest*, size	$P(dest) \leftarrow$ (New Block ID, 0, *size*, $\emptyset$)
MEMALLOC_HEAP *dest*, size	$P(dest) \leftarrow$ (New Block ID, 0, *size*, $\emptyset$)
REGCOPY *dest*, *src*	Copy all the content from register *src* to register *dest*
MEMCOPY *dest*, *src*	Copy all the Memory Chunks from region *src* to region *dest*
CALL @foo, *dest*, *arg*1, *arg*2	Create a new Stack Frame for @foo
RET *val*	(cont.) perform REGCOPY *dest*, *val*
RESUME *val*	store *val* for exception handler
LAND *dest*	(cont.) perform REGCOPY *dest*, *val*
RECV $1	$1 ← next runtime value
FDBIND *dest*	$T(dest) \leftarrow$ New File ID
FDWRITE *fd*, *data*	Log $T(fd)$ and $T(data)$ at sink
TAGTRACK #1, *target*	Log $T(target)$
TAGQUERY *target*, *tag*	$\begin{cases} \text{respond } 1, & \text{if } tag \in T(target) \\ \text{respond } 0, & \text{otherwise} \end{cases}$

Table 5: TSIR Mappings for Memory Access Operations

LLVM IR	TSIR
`%1 = alloca i32`	`MEMALLOC_STACK %1, 4`
`%1 = alloca i32, i32 4`	`MEMALLOC_STACK %1, 16`
`%1 = alloca [8 x i32]`	`MEMALLOC_STACK %1, 32`
`%2 = load i32, i32* %1`	`TAGCOPY %2, (*%1, 4)`
`%2 = load i32*, i32** %1`	`PTRCOPY %2, (*%1, 8)`
`store i32 2, i32* %1`	`TAGCLEAR %1`
`store i32 %2, i32* %1`	`TAGCOPY (*%1, 4), %2`
`store i32* %2, i32** %1`	`PTRCOPY (*%1, 8), %2`

Table 6: Runtime Logging Levels

Level	select	getelementptr
1	pointer only	-
2	pointer only	`int` and `char` arrays larger than 128 bytes are treated as singletons.
3 (default)	all	`int` and `char` arrays larger than 128 bytes are treated as singletons.
4	all	all

Finally, we optimize the TSIR by analyzing the definition and the use of local identifiers and runtime identifiers. Unused runtime values and TSIR instructions will be removed.

2.3 Taint Semantics Replay and Dynamic Information Flow Tracking

MIT decouples DIFT from program execution by breaking DIFT into a online record phase and an offline replay phase. During program execution, we only record the IDs of the basic blocks being executed and the runtime values. The actual DIFT takes place by replaying the TSIR of the target program according to the logs.

2.3.1 LLVM IR Instrumentation. The logging of basic block IDs and runtime values is implemented by instrumenting the LLVM IR from the target program. We place a call to the log function at the entry of each basic block. Basic blocks without explicit data flow are skipped during the instrumentation to reduce runtime logging overhead except that those referenced by a `phi` instruction should still be logged to correctly propagate the data flow for `TAGCOPY_PHI` and `PTRCOPY_PHI`.

For runtime value logging, for each reference to a LLVM IR local register by the TSIR's `RECV` instruction, a call to `log_rtvalue()` is placed right after it. If the value to be logged is a `vector`, the instrumentation will de-vectorizes it and inserts a separate logging call for each vector field. In general, the number of instrumented `log_rtvalue()` calls should match the number of `RECV` instructions in the TSIR.

2.3.2 TSIR Virtual Machine. By replaying the TSIR of the target program based on the basic block log and the runtime value log, the TSIR VM reconstructs the control flows and the information flows in the target program's execution. The architecture of TSIR VM is shown in Figure 1. The *Memory Block Table* tracks all the memory blocks allocated on the heap or on the stack. Instead of using per-byte shadow memory, we only maintain a memory block for each allocation. The *Global Variable Table* keeps track of all the global variables and their corresponding memory blocks. The *File Descriptor Table* stores the pseudo file descriptors and their corresponding tags. When replaying a `FDBIND` instruction, a pseudo file descriptor ID is chosen from the reserved tag value pool and gets bound with a virtual register. This allows as to set files as taint sources or taint sinks in DIFT.

To support multi-threaded programs, TSIR VM creates a *Thread Context* for each user thread and keeps it in the *Thread Context Table*. When replaying a basic block ID, TSIR VM first detects if a thread context switch is required. If the corresponding thread context does not exist, it will allocate a new one and switch from the current context to it.

When the control flow is transfered to a new function, a *Stack Frame* is allocated to maintain the function call states. The *Virtual Register Table* stores the states of virtual registers in the function scope. *Virtual Stack* tracks all the memory blocks allocated by the `MEMALLOC_STACK` instruction in the function scope. Finally, since runtime identifiers and runtime values have function scopes, a stack frame also includes a *Runtime Value Table* to track the mapping between the identifiers and the values.

During the replay of TSIR, the input basic block ID works like an instruction pointer. Given a basic block ID, the TSIR VM fetches all the TSIR instructions from the corresponding basic block and replays them. It tracks the usage of memory buffer and propagate the taint tags among the virtual registers and memory blocks.

When TSIR VM encounters a `TAGQUERY` instruction, it will check if the given region contains the tag of interest. The result will be passed back to the IFT runtime, which is blocked at a corresponding call to to `has_tag()` waiting for the check result. More details about the use of tag queries are covered in Section **??**.

2.3.3 Taint Propagation. The TSIR VM propagates the taint tags associated with the operands of TSIR instructions during the replay. The taint tags can either be encoded as a 32-bit integer or kept in a hash map. In the 32-bit integer encoding, each bit represents a tag so there can be up to 32 tags. The first 16 bits are reserved for virtual file descriptors and the last 16 bits are available for general uses.

3 EVALUATION

To confirm the applicability of MIT in production environment, we used a variety of target programs including common Unix command-line utilities, server programs, and benchmarks in SPEC CPU2017 to evaluate its runtime overhead. We also carried out a case study demonstrating the use of MIT for data provenance and anomaly detection in production environment. The experiments were conducted on a machine equipped with Intel Core i7-3770 processor and 16 GB of RAM, running Ubuntu Linux 16.04. Unless otherwise specified, we used the default log level 3. We always flushed the memory cache before each experiment run.

3.1 Runtime Overhead

Here we evaluate the runtime overhead by comparing the performances of the target programs with MIT instrumentation versus without MIT instrumentation. For command-line utilities and SPEC CPU benchmarks, the performance metric is the elapsed time for running a target program. For server programs, the performance metric is the number of served requests per second. The values presented in the evaluation results are the mean values with 95% confidential intervals from 5 repetitions of experiment runs.

3.1.1 Command-line Utilities. For this experiment, we chose five common Unix command-line utilities, which are GNU versions

of `wget`, `tar`, `gzip` and `md5sum`, `sha256sum` from `coreutils`. We ran `wget` to download the Linux kernel source compressed tarball (v4.12.10; 99.2MB). We used `tar` to archive the Linux kernel source tree (849MB). We later used `tar` to extract the Linux kernel source from the tarball (744MB). `md5sum` and `sha256sum` were invoked to compute the checksums of the tarball. `gzip` is used to compress the tarball, and later we ran gzip to decompress the compressed tarball.

Among them, `wget` and `tar` are I/O-bound, while `gzip` is CPU-bound. `md5sum` is both I/O-bound and CPU-bound, and `sha256sum` is more towards CPU-bound. The vanilla versions of the programs were compiled using clang 3.9 (*i.e.*, without MIT instrumentation).

The slowdowns due to MIT instrumentation are summarized in Figure 2. We can see that MIT instrumentation imposes negligible performance overhead on `wget`, `tar`, `md5sum` and `sha256sum`. However, for `gzip`, the slowdowns with respect to both the elapsed time and the CPU time are around 3x to 4x. This slowdown on CPU time indicates that the instrumented code is consuming a significant portion of the CPU time. This is because the execution of `gzip` involves frequent control flow switches.

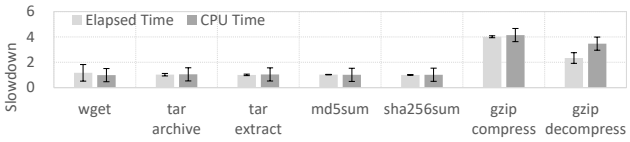

Figure 2: Slowdowns of Command-line Utilities

3.1.2 SPEC CPU. In the previous experiment, we observed that MIT can cause significant performance degradation for CPU-bound applications with intense control flow switches. To confirm the observation, we evaluated MIT against five CPU-bound benchmark programs from SPEC CPU 2017 including `mcf_s`, `omnetpp_s`, `leela_s`, `namd_r`, and `lbm_r`. The experiment results are presented in Figure 3. We can see that the CPU time slowdowns are around 2x-7x. For `mcf_s`, `omnetpp_s`, and `leela_s`, the elapsed time slowdowns are on par with the CPU time slowdowns. However, for `lbm_r` and `namd_R`, the elapsed time slowdowns jump to 13x and 25x respectively. This is because their log throughputs exceed the disk I/O throughput.

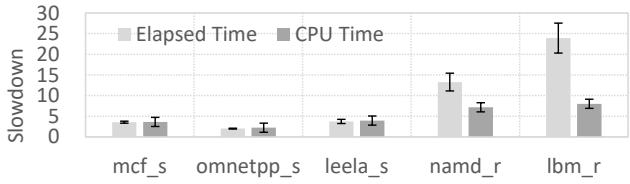

Figure 3: Slowdowns of SPEC CPU Benchmarks

3.1.3 Server Programs. While MIT exhibits noticeable performance degradation for CPU-bound applications with intense control flow switches, most of these applications are often used as a component in a software system. They have deterministic input/output relationships and there may not be much interest in knowing about how information flows within them in a real-world production environment. We believe the primary application targets for MIT would be server programs that constitute the dynamic information pathways in a real-world production system. In this experiment, we tested MIT with four server programs including `Lighttpd`, `Nginx`, `Redis`, and `Memcached`. For `Lighttpd` and `Nginx`, we use `wrk` [6] to generate 400 concurrent HTTP connection requests for a 10KB static file and evaluate the performance based on the number of completed requests per second. For `Redis`, we use the built-in utility `redis-benchmark` to measure the performance. For `Memcached`, we use `mcperf` [20] to measure the performance. Figure 4 illustrates the performance slowdowns for these server programs. Overall, the performance overheads are negligible. The worst case is with `Redis`, which has 17% overhead.

3.1.4 Log Level. MIT provides four log level settings for runtime value logging (Table 6). Here we used `gzip` from Section 3.1.1 to demonstrate how the log level settings affect the performance overhead. Figure 5 gives the elapsed time slowdown and the CPU time slowdown for each log level. We can see that the default log level 3 is a reasonable choice as it only incurs a little extra slowdown over level 1, and compared with level 4, the only trade-off is potential false-positives due to treating `char` and `int` arrays larger than 128 bytes as singletons.

3.2 Effectiveness of Information Flow Tracking

Here we demonstrate two potential applications of MIT. We set up a web server running `lighthttpd`. The `lighthttpd` program is instrumented by MIT. Based on the runtime logs and TSIR replay, we reconstruct the information flow to detect anomalies in the operation of the web server and also implement data provenance check on the outbound traffic.

3.2.1 Anomaly Detection. In this experiment, consider a possible misconfiguration in `lighttpd`, where a password-protected URL is mistakenly set as the homepage of the website. Clients will be able to access a sensitive HTML document at the protected URL freely without authentication by accessing the homepage URL (e.g. wget http://website/), even though access to the protected URL still requires password authentication (e.g. wget –user=USER –password=PASS http://website/ authorization_required.html).

Figure 6 is the information flow of accessing the sensitive HTML document via the homepage URL, while Figure 7 is the information flow of accessing the document via the password-proteted URL. The %6 variable in the function @connection_accept is tagged by `FDBIND` due to acceptance of incoming connection. The %5 variable in the function @http_chunk_append_file is also tagged by `FDBIND` due to opening the sensitive HTML document. Clearly, the two information flows are distinct. There is an additional flow going down from %6 in @connection_accept to the region encircled by dashed line in Figure 7, which corresponds to the password authentication process (*i.e.* @mod_auth_check_digest). Anomaly detection can be implemented by observing variations in the information flows, particularly the lack of flow through @mod_auth_check_digest when accessing the sensitive HTML file.

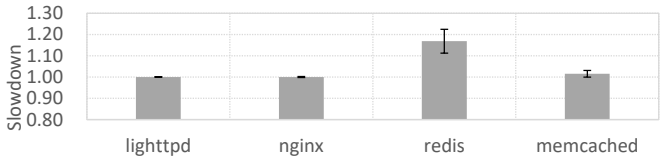

Figure 4: Slowdowns of Server Programs

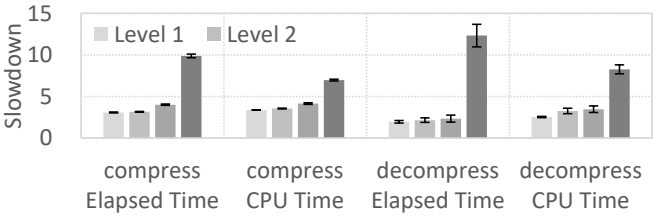

Figure 5: Slowdown of Different Log Levels

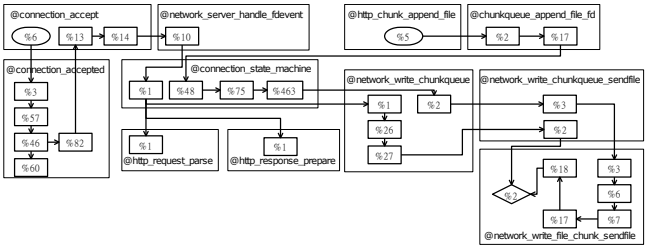

Figure 6: Information flow of accessing the sensitive HTML document via homepage URL

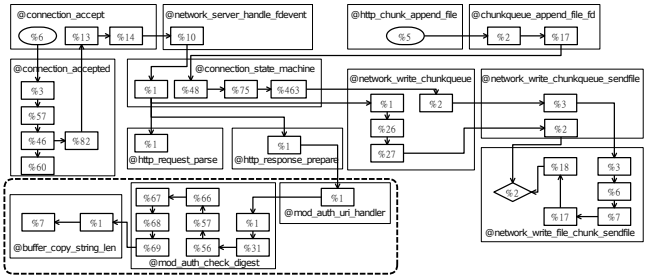

Figure 7: Information flow of accessing the sensitive HTML document via password-protected URL

3.2.2 Data Provenance Check. There are many existing work on building data provenance check for system objects based on system call interposition [8], Linux Audit system [5][2], Linux Security Module [19], or custom kernel modules [15][3]. However, the application processes are typically treated as as a black boxes. When the input/output relation of an application process is asynchronous or non-deterministic, these data provenance systems will not be able to precisely capture the causal relationships of data.

MIT can complement existing data provenance systems by exposing the information flow within an application process. Consider the scenario in Figure 6, the information flow from %5 in @http_chunk_append_file to %2 in @network_write_file_chunk_sendfile clearly establishes the causal relationship from the reading of a HTML file to the outbound transmission of the file.

4 RELATED WORK

IFA has been widely employed in detecting software bugs and vulnerabilities. It can detect buffer overflow attack, where malformed data spills onto the instruction pointer [21][14] or detect web application vulnerabilities that allow injection of malicious code [10]. Recently, it has also been applied to detect leaks of sensitive information in Android applications [7]. IFA falls short when the propagation of information flow depends on nondeterministic or external events. DIFT addresses the difficulties by tracking the actual information flow during program execution. Libdft [12] and TaintEraser [24] use dynamic binary instrumentation to embed the DIFT logic and shadow memory to maintain the taint tags. The external library handling in MIT is inspired by the function summaries in TaintEraser. Phosphor [4] provides DIFT for Java programs via bytecode instrumentation. GIFT [13] and DataFlowSanatizer [1] support DIFT of data variables in C program via instrumentation at the source code level and the LLVM IR level respectively.

The aforementioned systems interleaves DIFT with the application code. This can slow down program execution up to 20-30X [18]. The performance overhead can be reduced by employing a dedicated co-processor hardware for running DIFT [11]. And, for software-based solutions, Pirate [22] and ShadowReplica [9] pioneer the concept of decoupled DIFT, which reduces the performance overhead by offloading the information flow tracking logic onto other CPU cores. Within the application threads, the instrumented code only needs to record the control flows and the effective memory addresses. StraightTaint [16] and TaintPipe [17] avoid the need for effective memory address logging via symbolic taint analysis and a slight trade-off on the tracking accuracy. The use of symbolic taint analysis also offers the opportunity to parallelize the offline DIFT analysis. Distinct from existing work, MIT generates the TSIR at compile-time, which facilitate the application of IFA while it is equally capable of carrying out out DIFT under the same TSIR abstraction. Instead of relying on symbolic taint analysis, MIT achieves comparable runtime logging overhead by aligning the memory block bookkeeping mechanism with the memory allocation process of the compiler. Overall, both the design and the implementation are simpler and more intuitive to understand.

PoL-DFA [23] also supports decoupled DIFT by instrumenting control flow logging code in the LLVM IR of target program. Its carries out DIFT directly on the LLVM IR, while our TSIR allows a more streamlined IFA and DIFT process. Unlike our generic virtual register and memory block design, PoL-DFA keeps the taint tags with the program objects (*e.g.*, integers, structures, arrays). Furthermore, it aggressively considers arrays as singletons for taint propagation and does not log any runtime value.

5 CONCLUSION

We propose MIT to address the need for integrated IFA and DIFT in production environment. MIT derives the taint semantics TSIR from a target program at compile-time to facilitate IFA. The target program is lightly instrumented for the logging of control flows and runtime values during program execution. DIFT can be performed asynchronously by replaying the TSIR according to the runtime logs. By leveraging compile-time information, the memory block mechanism emulates the allocation and deallocation of memory variables efficiently, which greatly reduces the overhead of memory address logging.

The MIT prototype is built on top of LLVM. The IFT Runtime Library currently supports only C/C++ language, but it should be straightforward to extend the support for other languages that are compatible with LLVM. Our evaluation confirms the practicality of MIT in production environment. The runtime performance overhead is negligible for real-world server programs and I/O-bound applications. For CPU-bound applications, the performance overhead is on par with existing decoupled DIFT systems. The integration of IFA and DIFT in production systems enables a plethora of applications including data provenance check and information flow based anomaly detection. We believe the potential impacts of these applications and the use of MIT will be essential for making today's computing systems more dependable and more secure.

REFERENCES

[1] 2019. *LLVM DataFlowSanitizer*. Retrieved November 15, 2019 from http://clang.llvm.org/docs/DataFlowSanitizer.html

[2] Adam Bates, Devin J. Pohly, and Kevin R. B. Butler. 2016. *Secure and Trustworthy Provenance Collection for Digital Forensics*. Springer New York, New York, NY, 141–176. https://doi.org/10.1007/978-1-4939-6601-1_8

[3] Adam Bates, Dave Tian, Kevin R. B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-system Provenance for the Linux Kernel. In *Proceedings of the 24th USENIX Conference on Security Symposium (SEC'15)*. USENIX Association, Berkeley, CA, USA, 319–334. http://dl.acm.org/citation.cfm?id=2831143.2831164

[4] Jonathan Bell and Gail Kaiser. 2014. Phosphor: Illuminating Dynamic Data Flow in Commodity Jvms. *SIGPLAN Notices* 49, 10 (Oct. 2014), 83–101. https://doi.org/10.1145/2714064.2660212

[5] Ashish Gehani and Dawood Tariq. 2012. SPADE: Support for Provenance Auditing in Distributed Environments. In *Proceedings of the 13th International Middleware Conference (Middleware '12)*. Springer-Verlag New York, Inc., New York, NY, USA, 101–120. http://dl.acm.org/citation.cfm?id=2442626.2442634

[6] Will Glozer. [n.d.]. *wrk: Modern HTTP benchmarking tool*. https://github.com/wg/wrk

[7] Michael I. Gordon, Deokhwan Kim, Jeff H. Perkins, Limei Gilham, Nguyen Nguyen, and Martin C. Rinard. 2015. Information Flow Analysis of Android Applications in DroidSafe. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015*. https://www.ndss-symposium.org/ndss2015/information-flow-analysis-android-applications-droidsafe

[8] Matúš Harvan and Alexander Pretschner. 2009. State-Based Usage Control Enforcement with Data Flow Tracking Using System Call Interposition. In *Proceedings of the 2009 Third International Conference on Network and System Security (NSS '09)*. IEEE Computer Society, Washington, DC, USA, 373–380. https://doi.org/10.1109/NSS.2009.51

[9] Kangkook Jee, Vasileios P. Kemerlis, Angelos D. Keromytis, and Georgios Portokalidis. 2013. ShadowReplica: Efficient Parallelization of Dynamic Data Flow Tracking. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS '13)*. ACM, New York, NY, USA, 235–246. https://doi.org/10.1145/2508859.2516704

[10] N. Jovanovic, C. Kruegel, and E. Kirda. 2006. Pixy: a static analysis tool for detecting Web application vulnerabilities. In *2006 IEEE Symposium on Security and Privacy (SP'06)*. 6 pp.–263. https://doi.org/10.1109/SP.2006.29

[11] H. Kannan, M. Dalton, and C. Kozyrakis. 2009. Decoupling Dynamic Information Flow Tracking with a dedicated coprocessor. In *2009 IEEE/IFIP International Conference on Dependable Systems Networks*. 105–114. https://doi.org/10.1109/DSN.2009.5270347

[12] Vasileios P. Kemerlis, Georgios Portokalidis, Kangkook Jee, and Angelos D. Keromytis. 2012. Libdft: Practical Dynamic Data Flow Tracking for Commodity Systems. *SIGPLAN Notices* 47, 7 (March 2012), 121–132. https://doi.org/10.1145/2365864.2151042

[13] Lap Chung Lam and Tzi-cker Chiueh. 2006. A General Dynamic Information Flow Tracking Framework for Security Applications. In *Proceedings of the 22Nd Annual Computer Security Applications Conference (ACSAC '06)*. IEEE Computer Society, Washington, DC, USA, 463–472. https://doi.org/10.1109/ACSAC.2006.6

[14] David Larochelle and David Evans. 2001. Statically Detecting Likely Buffer Overflow Vulnerabilities. In *Proceedings of the 10th Conference on USENIX Security Symposium - Volume 10 (SSYM'01)*. USENIX Association, Berkeley, CA, USA, Article 14. http://dl.acm.org/citation.cfm?id=1251327.1251341

[15] Shiqing Ma, Xiangyu Zhang, and Dongyan Xu. 2016. ProTracer: Towards Practical Provenance Tracing by Alternating Between Logging and Tainting. In *23rd Annual Network and Distributed System Security Symposium, NDSS 2016, San Diego, California, USA, February 21-24, 2016*. http://wp.internetsociety.org/ndss/wp-content/uploads/sites/25/2017/09/protracer-towards-practical-provenance-tracing-alternating-logging-tainting.pdf

[16] Jiang Ming, Dinghao Wu, Jun Wang, Gaoyao Xiao, and Peng Liu. 2016. StraightTaint: Decoupled Offline Symbolic Taint Analysis. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE 2016)*. ACM, New York, NY, USA, 308–319. https://doi.org/10.1145/2970276.2970299

[17] Jiang Ming, Dinghao Wu, Gaoyao Xiao, Jun Wang, and Peng Liu. 2015. TaintPipe: Pipelined Symbolic Taint Analysis. In *Proceedings of the 24th USENIX Conference on Security Symposium (SEC'15)*. USENIX Association, Berkeley, CA, USA, 65–80. http://dl.acm.org/citation.cfm?id=2831143.2831148

[18] James Newsome and Dawn Song. 2005. Dynamic Taint Analysis for Automatic Detection, Analysis, and Signature Generation of Exploits on Commodity Software. In *Proceedings of the 12th Annual Network and Distributed System Security Symposium (NDSS '05)*.

[19] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: Collecting High-fidelity Whole-system Provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC '12)*. ACM, New York, NY, USA, 259–268. https://doi.org/10.1145/2420950.2420989

[20] Manju Rajashekhar and Clojure Janet. [n.d.]. *mcperf: A tool for measuring memcached server performance*. https://github.com/twitter/twemperf

[21] J. Viega, J. T. Bloch, Y. Kohno, and G. McGraw. 2000. ITS4: a static vulnerability scanner for C and C++ code. In *Proceedings 16th Annual Computer Security Applications Conference (ACSAC'00)*. 257–267. https://doi.org/10.1109/ACSAC.2000.898880

[22] Ryan Whelan, Tim Leek, and David Kaeli. 2013. Architecture-Independent Dynamic Information Flow Tracking. In *Proceedings of the 22Nd International Conference on Compiler Construction (CC'13)*. Springer-Verlag, Berlin, Heidelberg, 144–163. https://doi.org/10.1007/978-3-642-37051-9_8

[23] Gaoyao Xiao, Jun Wang, Peng Liu, Jiang Ming, and Dinghao Wu. 2016. Program-object Level Data Flow Analysis with Applications to Data Leakage and Contamination Forensics. In *Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy (CODASPY '16)*. ACM, New York, NY, USA, 277–284. https://doi.org/10.1145/2857705.2857747

[24] David (Yu) Zhu, Jaeyeon Jung, Dawn Song, Tadayoshi Kohno, and David Wetherall. 2011. TaintEraser: Protecting Sensitive Data Leaks Using Application-level Taint Tracking. *SIGOPS Operating Systems Review* 45, 1 (Feb. 2011), 142–154. https://doi.org/10.1145/1945023.1945039